Aidan Holloway

Senior Engine Programmer

Cary, NC • engineprog@gmail.com • +1 919-555-0162

PROFILE SUMMARY

- **Senior Engine Programmer** with 10 years of experience working on **a cross-platform AAA game engine** across **rendering, memory, job systems, and platform abstraction**, specializing in **render-graph design, custom allocators, and console-platform abstraction**.
- Solid technical background across engine context (**Unreal Engine 5**), languages (**C++20**, **C++17**), rendering APIs (**DirectX 12**, **Vulkan**), job systems on **fiber-based work-stealing scheduler**, physics on **PhysX 5**, target platforms **PS5, Xbox Series X|S, PC, and Switch**, and profiling with (**PIX**, **Tracy**), with strong fundamentals in **frame-budget discipline, data-oriented design, and platform-aware abstraction**.
- Deep expertise in **core engine architecture and ECS systems**, **custom allocators and cache-friendly data layout**, **render graphs and command-buffer management**, and **job systems and lock-free concurrency**, applying practices such as **data-oriented design and SIMD optimization** and **render-graph passes with fine-grained barriers** to deliver **frame-budget-clean, platform-portable, and team-friendly engine code**.
- Engaged collaborator working **cross-functionally** with **Rendering, Tools, Gameplay, and Platform** leadership in **multi-studio engine programs with quarterly stability cuts**, contributing to engine-architecture reviews, perf war rooms, and stability-cut retrospectives with a pragmatic, ship-it-then-polish mindset.
- Senior engineer who shares technical excellence and fosters a culture of **code-review rigor** and **frame-budget and memory-budget discipline** through code-review coaching and architecture mentorship, while running **weekly engine architecture council** sessions and authoring widely adopted engine-pattern and platform-abstraction handbooks.

TECHNICAL SKILLS

Languages & Build:	C++20, C++17, C, Rust (study), Python (tooling), CMake, premake, MSBuild, clang-tidy, sanitizers
Core Engine Systems:	Main loop, subsystem lifecycle, ECS, scene graph, reflection, serialization, message bus, event system, hot-reload
Memory & Performance:	Custom allocators (pool / stack / frame / slab), SIMD (SSE / AVX / NEON), cache-line layout, data-oriented design, fragmentation analysis
Rendering & Graphics APIs:	DirectX 12, Vulkan, Metal, render graphs, command-buffer recording, GPU resource lifetime, descriptor heaps, HLSL / GLSL shader compilation
Concurrency & Job Systems:	Fiber schedulers, work-stealing, lock-free queues (MPMC, SPSC), atomics, memory ordering, parallel-for primitives
Asset Pipeline & Streaming:	Asset importers (FBX, glTF), texture cookers (BC, ASTC), shader compilers, async loading, LOD systems, on-disk packaging
Platform & Console:	PS5 (Razor + GNM), Xbox Series X|S (PIX + DX12), Nintendo Switch (NVN), iOS / Android (Metal / Vulkan), TRC / TCR / Lotcheck
Tooling & Profiling:	Dear ImGui, Qt, custom editor frameworks, RenderDoc, PIX, RGP, Razor, Tracy, Superluminal, Intel VTune, Nsight

EDUCATION

North Carolina State University M.S. in Computer Science (Graphics & Systems) — Raleigh, NC • Sep 2014 - Jun 2016

WORK EXPERIENCE

Epic Games — Senior Engine Programmer

Cary, NC • Sep 2020 - Present

- Own engine architecture for the **Unreal Engine 5 core runtime and ECS** subsystem across **6** major engine releases over **4 years**, with consumers spanning **80+** licensee studios and **9** first-party titles on the engine roadmap.
- Designed a **tiered pool + slab** allocator with **per-frame arenas** that replaced the legacy **global new / delete** path, instrumented with sanitizer-friendly tagging; cut peak working-set by **184 MB** on **Series S** and reduced fragmentation by **38%**.
- Built render-graph passes on the **DirectX 12** and **Vulkan** backends, layering in **bindless descriptors** and **async-compute shadows** with fine-grained barriers and command-buffer reuse; delivered **2.3 ms** of GPU savings per frame on **Series X** across a **2-quarter** perf-pass window with no shader-permutation regressions.
- Designed the engine's **fiber-based work-stealing** job scheduler with **task priorities and dependency graphs**, scaling cleanly to **16** cores at **92%** CPU utilization on the flagship **Fortnite Battle Royale** benchmark across PC and Series X profiles.
- Built the **texture and mesh cooker** and the async **streaming layer** against **LOD streaming and on-disk packaging**, with priority queues and IO budgeting per zone; pulled cold-load times from **38 s** to **9 s** on **PS5 SSD** and unblocked the **first-party launch** milestone.
- Integrated the **Chaos physics solver** and the **animation-graph bridge** against the new **ECS scene model**, tuning broadphase / narrowphase budgets and pose-sampling caches; delivered a **44%** solver-time reduction on **dense ragdoll** scenes and held animation cost under **1.6 ms** across a **3-release** stabilization window.
- Built an in-engine **Tracy** bridge and **ImGui debug overlays** for the engine team and licensee studios; adopted by **32** engineers across **4** studios and cited in **3** quarterly engine reports as a **top-3** productivity win.

id Software — Engine Programmer

Richardson, TX • Jul 2016 - Aug 2020

- Owned the platform-abstraction layer for **idTech** covering **file I/O, threading, and graphics abstraction** across **PS4 / PS5 / Xbox One / Series / Switch**, delivering **zero cert findings** across **3** platform submissions.
- Drove engine maintenance and versioning on **idTech branch management** with **weekly** stability cuts, cutting nightly crash rate from **1.8%** to **0.3%** across the program.
- Mentored **4 mid-career engine** engineers through senior engine work, supporting **2** promotions to **Senior**; authored the team's **engine code-style and review handbook** adopted across **3** internal engine forks.
- Partnered with **Rendering, Tools, Gameplay, and Platform** teams across **2** shipped titles and **4** internal prototypes, authoring **34** architecture and integration RFCs that became the team's engine baseline; onboarded **6** new engine engineers into the branch and cert workflow.